Logistic Regression

From Linear Predictions to Probabilistic Classification

Lebnet Tech Fellows Program

Roadmap

The Classification Problem

Why Not Just Use Linear Regression?

Decision Boundaries

Choosing a Loss Function

Maximum Likelihood Interpretation

Computing the Gradient

Optimisation

Regularisation

Putting It All Together

From Regression to Classification

In linear regression we predict a **continuous** response $y \in \mathbb{R}$.

New question: What if y is **binary**?

$$y \in \{0, 1\}$$

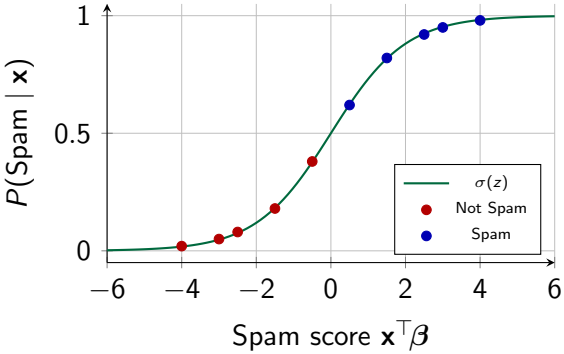
Training data:

$$D_n = \{(X_i, Y_i)\}_{1 \leq i \leq n}, \qquad \mathcal{X} = \mathbb{R}^d, \quad \mathcal{Y} = \{0, 1\}$$

Data points are i.i.d. from a distribution $\mathcal{P}$ on $\mathcal{X} \times \mathcal{Y}$.

Running Example: Email Spam Classification

- $Y = 1$: **spam**
- $Y = 0$: **not spam**
- Features $\mathbf{x}$: word frequencies, sender info, ...
- **Goal:** predict $P(\text{Spam} \mid \mathbf{x})$



Two Constraints a Probability Must Satisfy

We need a target function $p(\mathbf{x}) = P(Y = 1 \mid \mathbf{x})$.

Unlike regression, p is **not linear**. It must obey:

1. Boundedness

$0 \leq p(\mathbf{x}) \leq 1$ (it is a probability).

2. Diminishing Sensitivity

When $p(\mathbf{x}) \approx 1$, a small change in $\mathbf{x}$ should barely move p.

When $p(\mathbf{x}) \approx 0.5$, the same change has a much larger effect.

We need a **transformation** that turns a linear function into something satisfying both constraints.

The Logit Transformation

Idea: map p to the *log-odds*, bounded in both directions.

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right)$$

Set this equal to a linear predictor:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \mathbf{x}^\top \boldsymbol{\beta}$$

Hence the name **logistic regression**.

Deriving the Sigmoid Function

Solve for p:

$$\frac{p}{1-p} = e^{\beta_0 + \mathbf{x}^\top \boldsymbol{\beta}}$$

$$p\left(1 + e^{\beta_0 + \mathbf{x}^\top \boldsymbol{\beta}}\right) = e^{\beta_0 + \mathbf{x}^\top \boldsymbol{\beta}}$$

$$p(\mathbf{x}; \beta_0, \boldsymbol{\beta}) = \frac{1}{1 + e^{-(\beta_0 + \mathbf{x}^\top \boldsymbol{\beta})}}$$

This is the **sigmoid (logistic) function**:

$$\boxed{\sigma(z) = \frac{1}{1 + e^{-z}}}$$

From Probabilities to Predictions

Classification rule: predict $Y = 1$ when $p \geq 0.5$, i.e.

$$\beta_0 + \mathbf{x}^\top \boldsymbol{\beta} \geq 0.$$

Result: logistic regression gives a **linear classifier**.

Decision Boundary

The set $\{\mathbf{x} : \beta_0 + \mathbf{x}^\top \boldsymbol{\beta} = 0\}$ is a **hyperplane** in $\mathbb{R}^d$.

Finding the logistic regression parameters $\Leftrightarrow$ finding a linear classifier.

Distance from the Decision Boundary

Let $\mathbf{x}_0$ be any point on the boundary ($\boldsymbol{\beta}^\top \mathbf{x}_0 + \beta_0 = 0$).

The **signed distance** from $\mathbf{x}$ to the boundary:

$$\text{distance} = (\mathbf{x} - \mathbf{x}_0) \cdot \frac{\boldsymbol{\beta}}{|\boldsymbol{\beta}|} = \frac{\beta_0 + \mathbf{x}^\top \boldsymbol{\beta}}{|\boldsymbol{\beta}|}$$

Interpretation

Class probabilities depend on the **distance from the boundary**. Larger $|\boldsymbol{\beta}| \Rightarrow$ probabilities approach 0 or 1 more rapidly.

The 0-1 Loss: Simple but Intractable

Natural idea: count misclassifications.

$$\hat{\mathcal{R}}_n(\boldsymbol{\beta}) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}_{Y_i \neq \mathbf{1}_{X_i^\top \boldsymbol{\beta} \geq 0}}$$

Problem:

- **Not convex** in $\boldsymbol{\beta}$.
- **Not differentiable**.
- Minimisation is **NP-hard** in general.

We need a **convex surrogate**.

The Logistic Loss

Replace the 0-1 loss with a smooth, convex alternative:

$$\ell(\hat{y}, y) = y \log(1 + e^{-\hat{y}}) + (1 - y) \log(1 + e^{\hat{y}})$$

Logistic Regression Estimator

$$\hat{\boldsymbol{\beta}}_{\text{logit}} = \arg\min_{\boldsymbol{\beta} \in \mathbb{R}^d} \frac{1}{n} \sum_{i=1}^{n} \ell(\mathbf{X}_i^\top \boldsymbol{\beta}, Y_i)$$

Key advantage over hinge loss: the logistic loss has a **probabilistic interpretation** via $P(Y = 1 \mid X)$.

A Bernoulli Model

Model the conditional distribution:

$$P(Y = 1 \mid X = \mathbf{x}) = \sigma(\boldsymbol{\theta}^\top \mathbf{x})$$

$$P(Y = 0 \mid X = \mathbf{x}) = 1 - \sigma(\boldsymbol{\theta}^\top \mathbf{x})$$

Compact form:

$$P(Y = y \mid X = \mathbf{x}) = \sigma(\boldsymbol{\theta}^\top \mathbf{x})^y \left[1 - \sigma(\boldsymbol{\theta}^\top \mathbf{x})\right]^{1-y}$$

Log-Likelihood

For i.i.d. data the **log-likelihood** is:

$$LL(\boldsymbol{\theta}) = \sum_{i=1}^{n} \left[y^{(i)} \log \sigma(\boldsymbol{\theta}^\top \mathbf{x}^{(i)}) + (1 - y^{(i)}) \log(1 - \sigma(\boldsymbol{\theta}^\top \mathbf{x}^{(i)})) \right]$$

Logistic Loss = Negative Log-Likelihood

Using the identity $\sigma(-z) = 1 - \sigma(z)$:

$$-\log \sigma(z) = \log(1 + e^{-z})$$
$$-\log(1 - \sigma(z)) = \log(1 + e^{z})$$

Therefore:

$$\boxed{-LL(\boldsymbol{\theta}) = \sum_{i=1}^{n} \ell\big(\hat{y}^{(i)},\, y^{(i)}\big)}$$

Key Equivalence

Minimising the logistic loss $\iff$ Maximising the log-likelihood under a Bernoulli model.

Derivative of the Sigmoid

$$\frac{d}{dz}\sigma(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \sigma(z)\,(1-\sigma(z))$$

Chain rule for the parameters:

$$\frac{\partial}{\partial\theta_j}\sigma(\boldsymbol{\theta}^\top\mathbf{x}) = \sigma(\boldsymbol{\theta}^\top\mathbf{x})\,(1-\sigma(\boldsymbol{\theta}^\top\mathbf{x}))\,x_j$$

Gradient of the Log-Likelihood

For a **single** data point:

$$\frac{\partial LL}{\partial \theta_j} = \left[y - \sigma(\boldsymbol{\theta}^\top \mathbf{x})\right] x_j$$

Summing over the dataset:

$$\frac{\partial LL(\boldsymbol{\theta})}{\partial \theta_j} = \sum_{i=1}^{n} \left[y^{(i)} - \sigma(\boldsymbol{\theta}^\top \mathbf{x}^{(i)})\right] x_j^{(i)}$$

Matrix form:

$$\boxed{\nabla LL(\boldsymbol{\theta}) = \mathbf{X}^\top \big(\mathbf{y} - \sigma(\mathbf{X}\boldsymbol{\theta})\big)}$$

No Closed-Form Solution

Setting $\nabla \hat{\mathcal{R}}_n(\boldsymbol{\beta}) = 0$ yields a **non-linear** system with no closed-form solution.

Good news: the problem is **convex**.

$$\frac{\partial^2 \ell(\hat{y}, y)}{\partial \hat{y}^2} = \sigma(\hat{y})\,(1 - \sigma(\hat{y})) > 0$$

- Strictly convex $\Rightarrow$ **unique** minimum.
- Solvable by gradient descent/ascent, Newton's method, SGD.

Gradient Ascent Update Rule

To **maximise** the log-likelihood:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \nabla LL(\boldsymbol{\theta})$$

where $\alpha > 0$ is the learning rate.

- α too large $\Rightarrow$ oscillation
- α too small $\Rightarrow$ slow convergence
- Iterate until $\|\Delta\boldsymbol{\theta}\| < \varepsilon$

Preventing Overfitting

When $d > n$ (more features than samples), logistic regression can **overfit**.

Fix: add an L2 penalty.

L2-Regularised Logistic Regression

$$\hat{\boldsymbol{\beta}}_{\text{ridge}} = \underset{\boldsymbol{\beta} \in \mathbb{R}^d}{\arg\min} \left\{ \frac{1}{n} \sum_{i=1}^{n} \ell(\mathbf{X}_i^\top \boldsymbol{\beta}, Y_i) + \lambda \|\boldsymbol{\beta}\|_2^2 \right\}$$

$\lambda > 0$ controls the bias-variance trade-off:

- Larger $\lambda \Rightarrow$ more shrinkage, less overfitting.
- Smaller $\lambda \Rightarrow$ closer to unregularised fit.

The Training Algorithm

Logistic Regression via Gradient Ascent

1. **Input:** $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$, learning rate η
2. Initialise $\boldsymbol{\theta} \leftarrow \mathbf{0}$
3. **Repeat:**
   3a. Compute gradient: $\nabla LL = \mathbf{X}^\top(\mathbf{y} - \sigma(\mathbf{X}\boldsymbol{\theta}))$
   3b. Update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta \nabla LL$
4. **Until** $\|\Delta\boldsymbol{\theta}\| < \varepsilon$
5. **Output:** $\boldsymbol{\theta}$

Recap: The Full Story

1. **Problem:** predict a binary outcome from features.
2. **Model:** the logit transform makes probabilities linear $\Rightarrow$ sigmoid function.
3. **Geometry:** the decision boundary is a hyperplane.
4. **Loss:** the 0-1 loss is intractable; the logistic loss is convex *and* equals the negative log-likelihood.
5. **Gradient:** $\nabla LL = \mathbf{X}^\top(\mathbf{y} - \sigma(\mathbf{X}\boldsymbol{\theta}))$.
6. **Optimisation:** gradient ascent converges to the unique solution.
7. **Regularisation:** L2 penalty prevents overfitting.

Questions?